

Part I: Basic Loop Complexities

Question 1: Determine the time complexity

```
for(int i=0; i<n; i++) {  
    cout<<i;  
}
```

Answer: $O(n)$

Explanation: A single linear loop where the control variable i increments by 1 in each step and runs exactly n times.

Question 2: Determine the time complexity

```
for(int i=0; i<n; i++) {  
    for(int j=0; j<n; j++) {  
        cout<<"*";  
    }  
}
```

Answer: $O(n^2)$

Explanation: Two fully nested loops where both the outer loop and the inner loop iterate independently n times, resulting in $n \times n = n^2$ total operations.

Question 3: Determine the time complexity

```
for(int i=0; i<n; i++) {  
    for(int j=0; j<i; j++) {  
        cout<<"*";  
    }  
}
```

Answer: $O(n^2)$

Explanation: The inner loop depends on the current value of i . The total number of executions is the arithmetic series: $0 + 1 + 2 + \dots + (n-1) = n(n-1)/2$, which simplifies asymptotically to $O(n^2)$.

Question 4: Determine the time complexity

```
for(int i=1; i<n; i*=2) {  
    cout<<i;  
}
```

```
}
```

Answer: $O(\log n)$

Explanation: The loop control variable i doubles in every step ($i *= 2$), meaning the number of iterations grows logarithmically with respect to n .

Question 5: Determine the time complexity

```
for(int i=n; i>1; i/=2) {  
    cout<<i;  
}
```

Answer: $O(\log n)$

Explanation: The loop starts at n and is divided by 2 during each iteration step ($i /= 2$) until it drops to 1, taking logarithmic time.

Question 6: Determine the time complexity

```
for(int i=1; i<=n; i++) {  
    for(int j=1; j<=log(n); j++) {  
        cout<<"*";  
    }  
}
```

Answer: $O(n \log n)$

Explanation: The outer loop runs linearly n times. For every iteration, the inner loop runs a logarithmic number of times ($\log n$). Multiplying their bounds gives $O(n \log n)$.

Question 7: Determine the time complexity

```
while(n>0) {  
    n--;  
}
```

Answer: $O(n)$

Explanation: A classic linear sequence where n decrements step-by-step down to zero.

Question 8: Determine the time complexity

```
while(n>1) {  
    n/=2;  
}
```

Answer: $O(\log n)$

Explanation: The variable n is systematically halved in each step of the loop until it hits 1.

Part II: Consecutive and Multi-variable Loops

Question 9: Determine the time complexity

```
for(int i=1; i<=n; i++) { }  
for(int j=1; j<=n; j++) { }
```

Answer: $O(n)$

Explanation: Two consecutive independent loops. The time is additive rather than multiplicative: $n + n = 2n$. Dropping constant factors results in $O(n)$.

Question 10: Determine the time complexity

```
for(int i=1; i<=n; i++) { }  
for(int j=1; j<=n*n; j++) { }
```

Answer: $O(n^2)$

Explanation: The loops are consecutive. Total runtime is $n + n^2$. In asymptotic notation, we only keep the highest-order dominant term, which is $O(n^2)$.

Question 11: Determine the time complexity

```
for(int i=1; i<=n; i++) {  
    for(int j=1; j<=n*n; j++) { }  
}
```

Answer: $O(n^3)$

Explanation: These loops are nested. The outer loop runs n times and the inner loop runs n^2 times for each outer pass. Multiplied together, they yield $n \times n^2 = n^3$.

Question 12: Determine the time complexity

```
for(int i=1; i<=n; i*=3) { }
```

Answer: $O(\log_3 n) = O(\log n)$

Explanation: The control index triples at every step ($i *= 3$). The base of the logarithm changes to 3, but in Big O notation, base changes are constants and are dropped.

Question 13: Determine the time complexity

```
for(int i=1; i<=n; i++) {  
    for(int j=1; j<=i*i; j++) { }
```

```
}
```

Answer: $O(n^3)$

Explanation: The inner loop runs i^2 times. Summing across all iterations: $1^2 + 2^2 + 3^2 + \dots + n^2 = n(n+1)(2n+1)/6$, which establishes an asymptotic bound of $O(n^3)$.

Part III: Recursive Function Complexities

Question 14: Determine the time complexity

```
void fun(int n) {  
    if(n==0) return;  
    fun(n-1);  
}
```

Answer: $O(n)$

Explanation: Each call decrements n by 1, yielding a linear call stack sequence with exactly n total execution steps.

Question 15: Determine the time complexity

```
void fun(int n) {  
    if(n<=1) return;  
    fun(n/2);  
}
```

Answer: $O(\log n)$

Explanation: The input size is divided by 2 at each step of recursion, resolving to a total depth bounded logarithmically.

Question 16: Determine the time complexity

```
void fun(int n) {  
    if(n==0) return;  
    fun(n-1);  
    fun(n-1);  
}
```

Answer: $O(2^n)$

Explanation: Every execution branches directly into two subsequent recursive calls. This builds a complete binary tree of recursive execution paths with a total node count of $2^{n+1} - 1$.

Question 17: Determine the time complexity

```
int fib(int n) {  
    if(n<=1) return n;  
    return fib(n-1)+fib(n-2);  
}
```

Answer: $O(2^n)$

Explanation: Standard recursive Fibonacci calculations branch twice per level. A strict tight mathematical analysis yields the golden ratio order $O(1.618^n)$, bounded generally within $O(2^n)$.

Question 18: Determine the time complexity

```
int fact(int n) {  
    if(n<=1) return 1;  
    return n*fact(n-1);  
}
```

Answer: $O(n)$

Explanation: Standard mathematical factorial recursion executing exactly n times sequentially.

Question 19: Determine the time complexity

```
void fun(int n) {  
    if(n<=0) return;  
    fun(n-1);  
    cout<<n;  
}
```

Answer: $O(n)$

Explanation: The print operation takes $O(1)$ time. The execution structure remains identical to linear recursion containing n calls.

Question 20: Determine the time complexity

```
void fun(int n) {  
    if(n<=1) return;  
    fun(n/2);  
    cout<<n;  
}
```

Answer: $O(\log n)$

Explanation: The problem size is halved at each stage. The print statement takes constant time per level, leading to logarithmic complexity.

Part IV: Recurrence Relations

Question 21: Solve the recurrence relation

$$T(n) = T(n-1) + 1$$

Answer: $O(n)$

Proof via Substitution:

- $T(n) = T(n-1) + 1$
- $T(n) = [T(n-2) + 1] + 1 = T(n-2) + 2$
- Expanding this k times gives: $T(n) = T(n-k) + k$
- Let $n-k = 0 \rightarrow k = n$. Thus, $T(n) = T(0) + n = O(n)$

Question 22: Solve the recurrence relation

$$T(n) = T(n-1) + n$$

Answer: $O(n^2)$

Proof via Substitution:

- $T(n) = T(n-1) + n$
- $T(n) = [T(n-2) + (n-1)] + n = T(n-2) + n + (n-1)$
- Unfolding all steps completely yields the sum: $n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2 = O(n^2)$

Question 23: Solve the recurrence relation

$$T(n) = T(n/2) + 1$$

Answer: $O(\log n)$

Proof via Master Theorem:

- Format: $T(n) = aT(n/b) + \Theta(n^c)$, where $a = 1$, $b = 2$, $c = 0$
- Calculate $\log_b(a) = \log_2(1) = 0$. Since $c = \log_b(a)$, Case 2 applies:
- $T(n) = \Theta(n^c \log n) = O(\log n)$

Question 24: Solve the recurrence relation

$$T(n) = 2T(n-1) + 1$$

Answer: $O(2^n)$

Proof via Substitution:

- $T(n) = 2T(n-1) + 1$
- $T(n) = 2[2T(n-2) + 1] + 1 = 4T(n-2) + 2 + 1$
- Expanding k times results in: $T(n) = 2^k T(n-k) + (2^{k-1} + 2^{k-2} + \dots + 1)$

- Setting $k = n$ yields a geometric series summing up to $2^n - 1 = O(2^n)$

Question 25: Solve the recurrence relation

$$T(n) = T(n/2) + n$$

Answer: $O(n)$

Proof via Master Theorem:

- Here $a = 1$, $b = 2$, $c = 1$. Calculate $\log_b(a) = \log_2(1) = 0$
- Since $c > \log_b(a)$ ($1 > 0$), Master Theorem Case 3 applies:
- $T(n) = \Theta(n^c) = O(n)$

Part V: Bonus GATE-Level Questions

Question 26: Determine the time complexity

```
for(int i=1; i<=n; i++) {
    for(int j=1; j<=i; j*=2) { }
```

Answer: $O(n \log n)$

Explanation: The inner loop runs $\log_2(i)$ times for a given i . Summing across all items: $\sum \log_2(i) = \log_2(1) + \log_2(2) + \dots + \log_2(n) = \log_2(n!)$. By Stirling's Approximation, $\log_2(n!) = O(n \log n)$.

Question 27: Determine the time complexity

```
for(int i=1; i<=n; i*=2) {
    for(int j=1; j<=n; j++) { }
```

Answer: $O(n \log n)$

Explanation: The outer loop is logarithmic ($\log n$ steps) because i doubles each round. The inner loop executes a fixed n times independently. Combining them results in $O(n \log n)$.

Question 28: Determine the time complexity

```
for(int i=n; i>=1; i--) {
    for(int j=i; j>=1; j--) { }
```

Answer: $O(n^2)$

Explanation: This operates backwards but follows an identical structural pattern to Q3. The total step count is a descending series: $n + (n-1) + (n-2) + \dots + 1 = n(n+1)/2 = O(n^2)$.

Question 29: Determine the time complexity

```
for(int i=1; i<=n; i++) {  
    for(int j=1; j<=n; j*=2) { }  
}
```

Answer: $O(n \log n)$

Explanation: The outer loop runs exactly n times. The inner loop is independent and executes $\log n$ times because j doubles each step. The overall total is $O(n \log n)$.

Question 30: Determine the time complexity

```
void fun(int n) {  
    if(n<=1) return;  
    fun(n/2);  
    fun(n/2);  
}
```

Answer: $O(n)$

Explanation: The recurrence relation is written as: $T(n) = 2T(n/2) + O(1)$. Using Master Theorem: $a = 2$, $b = 2$, $c = 0$. We evaluate $\log_b(a) = \log_2(2) = 1$. Since $\log_b(a) > c$ ($1 > 0$), Case 1 applies, yielding $O(n^{\log_b a}) = O(n)$.